TerraPrice

Multi-Cloud Cost Intelligence Platform

Technical Document

Author	Abbu Talha
Project	TerraPrice
Version	1.0
Date	March 2026
Status	Deployed & running
Cloud Providers	AWS | Azure | GCP
GitHub	https://github.com/AbbuTalha8585/Terraprice

1. Project Description

1.1 Overview

TerraPrice is a full-stack, cloud-native web application designed to provide real-time multi-cloud cost intelligence. It enables developers, DevOps engineers, and cloud architects to compare, forecast, and optimize infrastructure costs across Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP) from a single unified interface.

The platform bridges the gap between infrastructure-as-code (IaC) definitions and actual cloud spend by parsing Terraform HCL files, fetching live pricing data, detecting cost inefficiencies, and providing AI-powered Terraform code generation.

1.2 Problem Statement

Cloud infrastructure costs are complex, fragmented, and constantly changing. Teams using multiple cloud providers struggle with:

- No single view of pricing across AWS, Azure, and GCP
- Inability to forecast 12-month infrastructure spend before provisioning
- Difficulty identifying overprovisioned or cost-inefficient resources
- Manual effort required to translate business requirements into Terraform code
- Lack of real-time monitoring for deployed infrastructure costs

1.3 Solution — TerraPrice Features

Live Multi-Cloud Pricing

Fetches real-time pricing data from AWS, Azure, and GCP APIs. Users can compare compute, storage, and networking costs side-by-side for equivalent resource types across all three providers.

HCL / Terraform File Parser

Accepts uploaded or pasted Terraform HCL files and automatically extracts resource definitions. Each resource is matched to live cloud pricing, providing an instant cost breakdown of the entire infrastructure plan.

Infrastructure Smell Detector

Analyzes Terraform configurations and deployed resources to identify cost anti-patterns such as oversized instance types, unattached EBS volumes, idle load balancers, and missing auto-scaling policies.

Spot Score Calculator

Evaluates workloads and recommends AWS Spot Instance usage with a calculated savings score, helping teams reduce compute costs by up to 90% for interruption-tolerant workloads.

12-Month Cost Forecasting

Generates a forward-looking cost forecast based on current resource usage patterns, pricing trends, and user-defined growth parameters. Outputs are visualized as interactive charts on the dashboard.

Disaster Recovery (DR) Mode

Calculates the cost of running a full DR environment across multiple cloud regions and providers, helping teams plan business continuity budgets accurately.

AI-Powered Terraform Generator

Uses the Anthropic Claude API to convert natural language descriptions into production-ready Terraform code. Users describe their infrastructure in plain English and receive complete, validated HCL output.

1.4 Project Goals

1. Demonstrate full-stack development competency across frontend, backend, and infrastructure layers
2. Deploy a production-grade application on Kubernetes with full CI/CD automation
3. Integrate live APIs from three major cloud providers
4. Implement observability and monitoring with Grafana dashboards
5. Showcase AI-augmented developer tooling using the Anthropic Claude API

1.5 Deployment Summary

Parameter	Value
Cloud Provider	Amazon Web Services (AWS)
Region	us-east-1 (N. Virginia)
Container Orchestration	Amazon EKS (Elastic Kubernetes Service)
Infrastructure Provisioning	Terraform v1.14.4
Container Registry	Amazon ECR (Elastic Container Registry)
Database	Amazon RDS — PostgreSQL 15

Parameter	Value
Cache Layer	Redis (deployed in EKS)
Object Storage	Amazon S3
CI/CD Pipeline	GitHub Actions
Monitoring	Grafana Cloud + Grafana Alloy
GitHub Repository	https://github.com/AbbuTalha8585/Terraprice
Status	Fully Deployed and Running

2. Architecture

2.1 High-Level Architecture Overview

TerraPrice follows a modern cloud-native microservices architecture. The system is divided into three primary layers: the Presentation Layer (React frontend), the Application Layer (FastAPI backend), and the Infrastructure Layer (AWS services, Kubernetes, and databases).

2.2 Architecture Layers

Layer 1 — Presentation Layer (Frontend)

- Built with React 18 and Vite for fast development and optimized production builds
- Communicates with the backend via RESTful API calls over HTTPS
- Deployed as a containerized service inside the EKS cluster
- Served behind an AWS Load Balancer (ALB Ingress Controller)

Layer 2 — Application Layer (Backend)

- Python FastAPI application exposing REST endpoints for all features
- Handles HCL parsing, pricing API integration, cost calculations, and AI generation
- Communicates with PostgreSQL for persistent data and Redis for caching
- Deployed as a Kubernetes Deployment with horizontal pod autoscaling (HPA)

Layer 3 — Data Layer

- PostgreSQL 15 on Amazon RDS — stores user data, pricing history, and forecasts
- Redis — in-memory cache for live pricing data to reduce external API calls
- Amazon S3 — stores cost reports, exported Terraform files, and static assets

Layer 4 — External Integrations

- AWS Pricing API — real-time EC2, RDS, S3, and Lambda pricing
- Azure Retail Prices API — real-time Azure VM and services pricing
- GCP Cloud Billing API — real-time GCP Compute Engine and services pricing
- Anthropic Claude API — AI-powered natural language to Terraform generation

Layer 5 — Infrastructure & DevOps Layer

- Terraform provisions all AWS resources (VPC, EKS, RDS, S3, ECR, IAM)
- Docker containers built for both frontend and backend services
- GitHub Actions CI/CD pipeline automates build, test, and deployment
- Grafana Cloud dashboards provide real-time monitoring and HPA visualization

2.3 Network Architecture

All resources are deployed inside a dedicated AWS VPC with public and private subnets across two Availability Zones for high availability.

Resource	Subnet Type	Details
ALB / Ingress	Public	Receives public internet traffic
EKS Worker Nodes	Private	Runs frontend, backend, Redis pods
Amazon RDS	Private	Multi-AZ PostgreSQL, no public access

2.4 Architecture Diagram

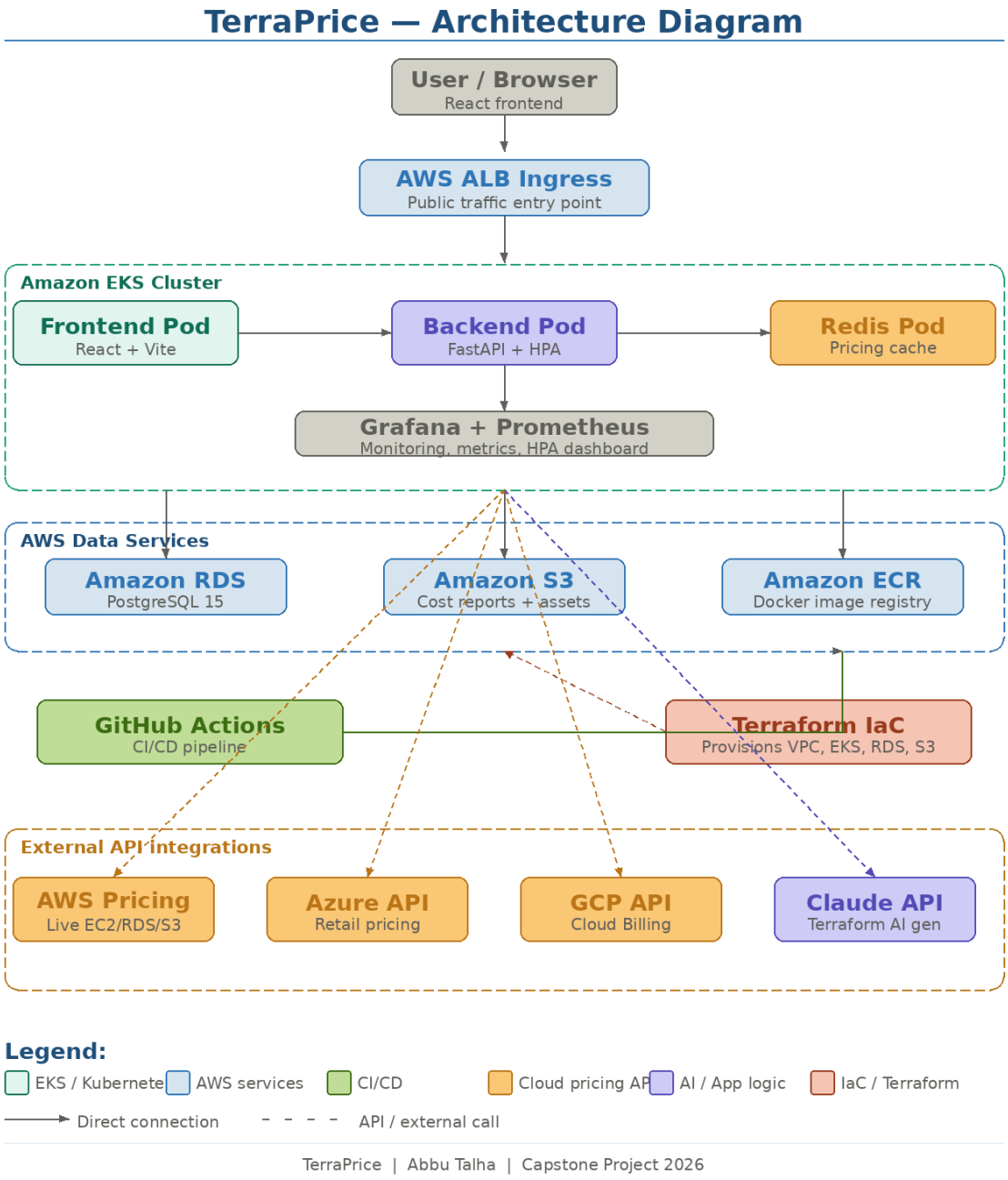

Figure 1: TerraPrice Full System Architecture

Diagram key: Teal = EKS pods | Blue = AWS services | Green = CI/CD | Amber = Cloud pricing APIs | Purple = AI/App logic | Coral = IaC/Terraform. Solid arrows = direct connections. Dashed arrows = external API calls.

3. Components & Services

3.1 Component Overview Table

Component	Type	Description	Technology
Frontend UI	Application	Single-page application providing dashboards, pricing tables, HCL upload, and forecast charts	React 18 + Vite
Backend API	Application	REST API server handling all business logic, pricing integration, and AI generation	Python FastAPI
HCL Parser	Module	Parses uploaded Terraform files and extracts resource type, size, and region info	Python + hcl2
Pricing Engine	Module	Fetches and normalizes live pricing from AWS, Azure, and GCP APIs	Python + REST APIs
Smell Detector	Module	Identifies cost inefficiencies in Terraform configs and live resources	Python rules engine
Spot Calculator	Module	Calculates potential savings from switching to spot/preemptible instances	Python
Forecast Engine	Module	Projects 12-month costs using usage trends and configurable growth rates	Python + NumPy
DR Mode	Module	Estimates cost of full disaster recovery environment across regions/clouds	Python
AI Generator	Module	Converts natural language to Terraform HCL using Claude API	Anthropic Claude API
PostgreSQL DB	Database	Stores users, pricing history, forecasts, and saved configurations	Amazon RDS PostgreSQL 15
Redis Cache	Cache	Caches live pricing data to minimize external API calls	Redis (in EKS)

Component	Type	Description	Technology
S3 Bucket	Storage	Stores generated cost reports, exported Terraform files, and assets	Amazon S3
EKS Cluster	Infrastructure	Managed Kubernetes cluster running all application pods	Amazon EKS
ECR Registry	Infrastructure	Private Docker image registry for backend and frontend images	Amazon ECR
Terraform IaC	Infrastructure	Provisions all AWS infrastructure including VPC, EKS, RDS, ECR, IAM, S3	Terraform v1.14.4
CI/CD Pipeline	DevOps	Automates build, test, Docker push to ECR, and Kubernetes deployment	GitHub Actions
Grafana	Monitoring	Real-time dashboards for app metrics, pod health, and HPA scaling events	Grafana Cloud + Alloy

3.2 AWS Services Used

AWS Service	Resource ID / Name	Purpose
Amazon VPC	vpc-0c2ed644fc2111c4c	Isolated network for all TerraPrice resources
Amazon EKS	terraprice-eks-cluster	Managed Kubernetes cluster for all pods
Amazon RDS	PostgreSQL 15	Primary application database
Amazon ECR	terraprice-backend / frontend	Docker image registry for CI/CD pipeline
Amazon S3	terraprice-cost-reports-e0109aec	Cost report storage and static file assets
IAM	terraprice-user	Access management for Terraform and CI/CD
Subnets	subnet-030437dc9e72a6f93 /	Public and private subnets across AZs

AWS Service	Resource ID / Name	Purpose
	subnet-026690ab833da56ad	

3.3 External API Integrations

- AWS Pricing API (pricing.us-east-1.amazonaws.com) — EC2, RDS, S3, Lambda pricing
- Azure Retail Prices API (prices.azure.com) — Azure VM, storage, networking pricing
- GCP Cloud Billing Catalog API — Compute Engine, Cloud Storage, GKE pricing
- Anthropic Claude API (api.anthropic.com) — Natural language to Terraform HCL generation

3.4 Technology Stack Summary

Layer	Technology	Version / Details
Frontend	React + Vite	React 18, Vite 5
Backend	Python FastAPI	Python 3.11, FastAPI 0.100+
Database	PostgreSQL	v15 on Amazon RDS
Cache	Redis	Latest stable, in-cluster
Container	Docker	Multi-stage builds
Orchestration	Kubernetes / EKS	AWS EKS (managed)
IaC	Terraform	v1.14.4
CI/CD	GitHub Actions	YAML workflows
Monitoring	Grafana Cloud + Alloy	abbutalha1001.grafana.net
AI Integration	Anthropic Claude API	claude-sonnet model
Cloud — Primary	Amazon Web Services	us-east-1
Cloud — Pricing	Azure + GCP	Pricing API integration only

4. Screenshots

The following screenshots demonstrate TerraPrice running live on AWS EKS, accessible at the public load balancer URL.

4.1 Dashboard — Multi-Cloud Pricing Overview

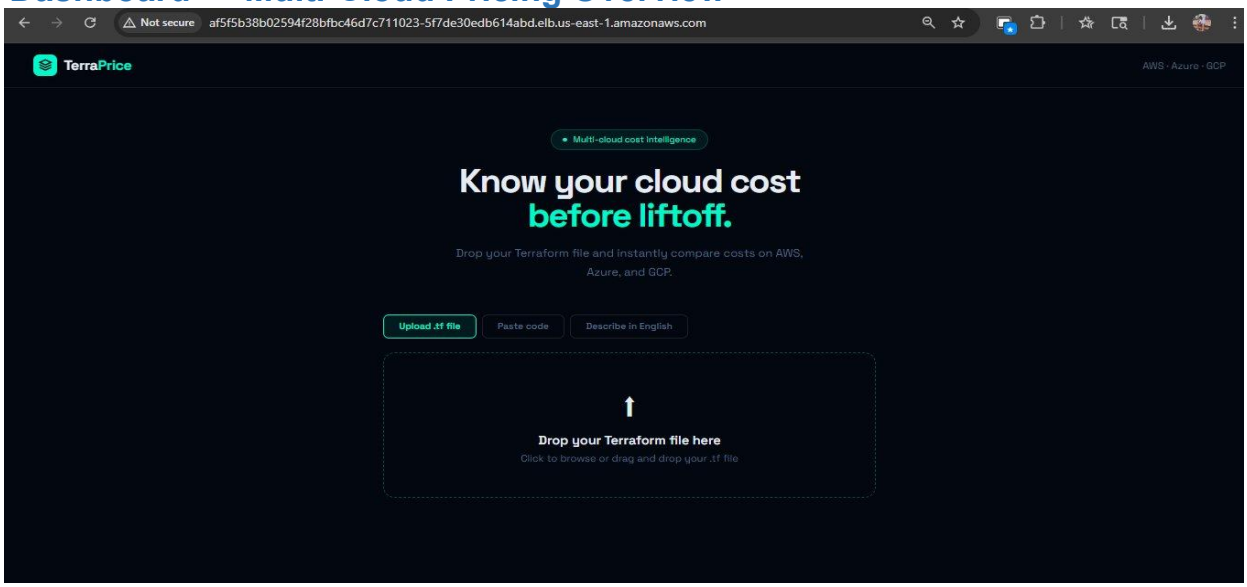

Figure 2: TerraPrice homepage — Upload .tf file, Paste code, or Describe in English

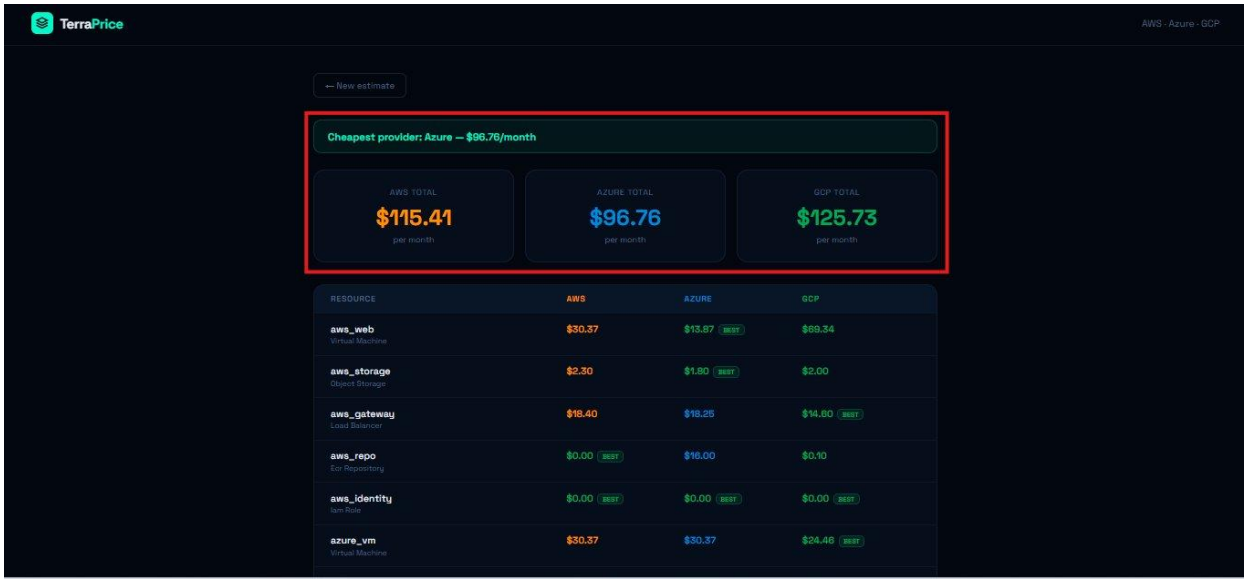

Figure 3: Live multi-cloud pricing comparison — AWS $115.41 | Azure $96.76 (BEST) | GCP $125.73

4.2 HCL File Upload & Cost Breakdown

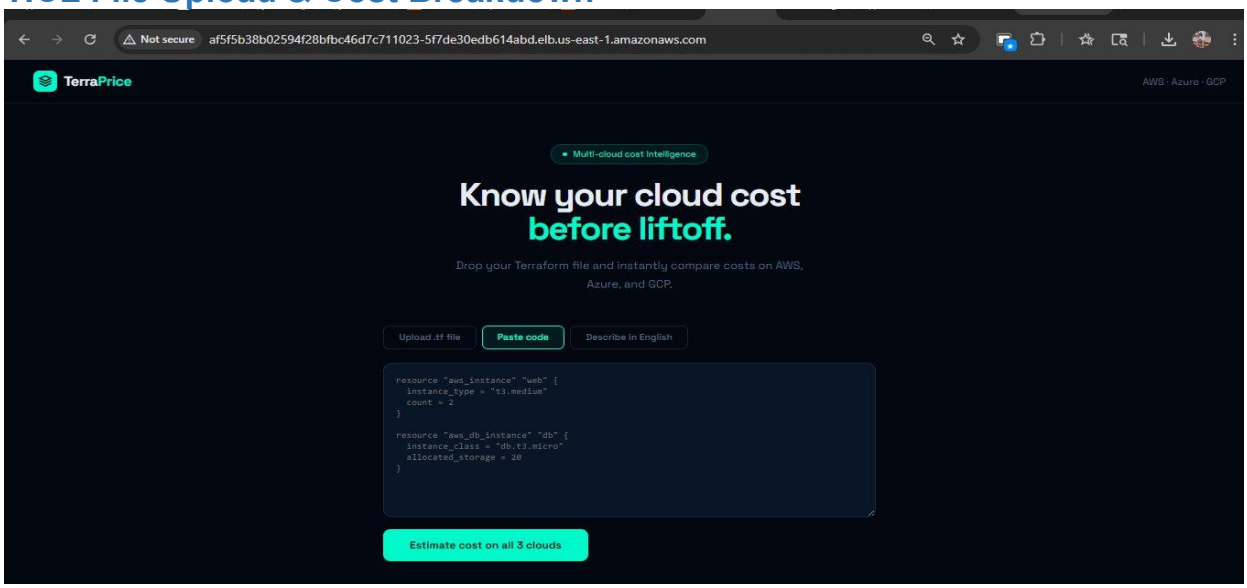

Figure 4: Paste Terraform HCL code and click "Estimate cost on all 3 clouds"

RESOURCE	AWS	AZURE	GCP
aws_web Virtual Machine	$30.37	$13.87 BEST	$69.34
aws_storage Object Storage	$2.30	$1.80 BEST	$2.00
aws_gateway Load Balancer	$18.40	$18.25	$14.80 BEST
aws_repo Ecr Repository	$0.00 BEST	$18.00	$0.10
aws_identity Iam Role	$0.00 BEST	$0.00 BEST	$0.00 BEST
azure_vm Virtual Machine	$30.37	$30.37	$24.48 BEST
azure_blob Object Storage	$2.30	$1.80 BEST	$2.00
gcp_node Virtual Machine	$30.37	$13.87	$12.23 BEST
gcp_bucket Object Storage	$2.30	$1.80 BEST	$2.00
hsm_cluster Cloudhsm V2 Cluster	Not Available	Not Available	Not Available
Total / month	$115.41	$96.76	$125.73

Figure 5: Detailed cost breakdown per resource across AWS, Azure, and GCP with BEST price highlighted

4.3 Infrastructure Smell Detector Results

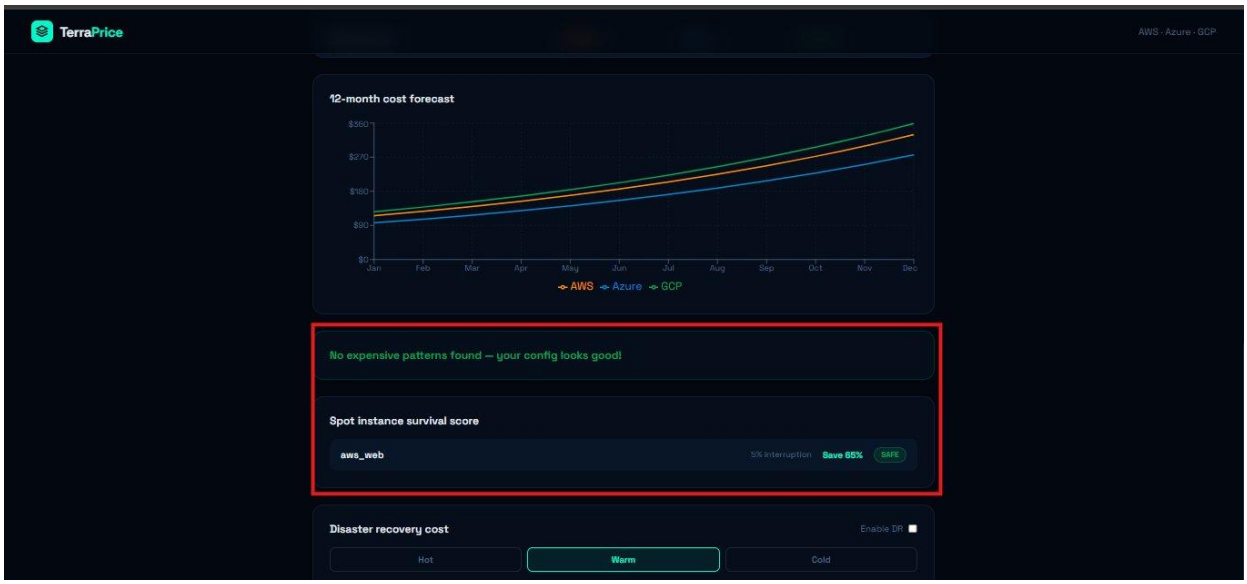

Figure 6: Infrastructure smell detector — "No expensive patterns found" + Spot instance survival score showing SAFE with 65% savings

4.4 Spot Score Calculator

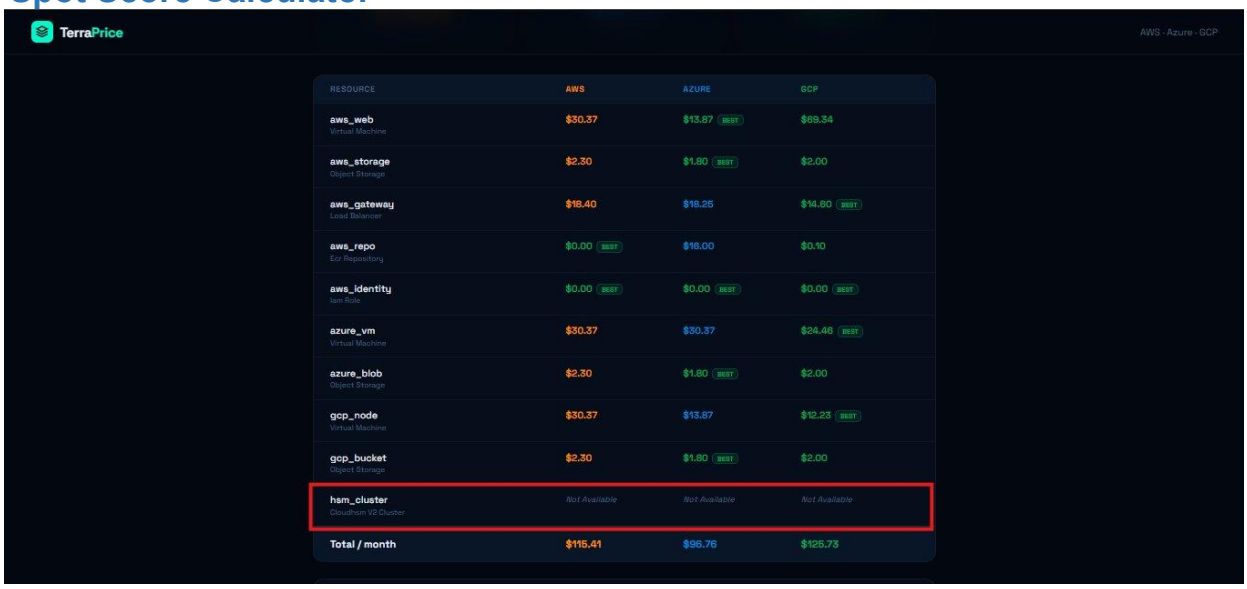

Figure 7: Spot instance survival score — aws_web: 5% interruption rate, Save 65%, rated SAFE

4.5 12-Month Cost Forecast Chart

Figure 8: 12-month cost forecast showing AWS, Azure, and GCP projected costs from Jan to Dec

4.6 DR Mode Cost Estimate

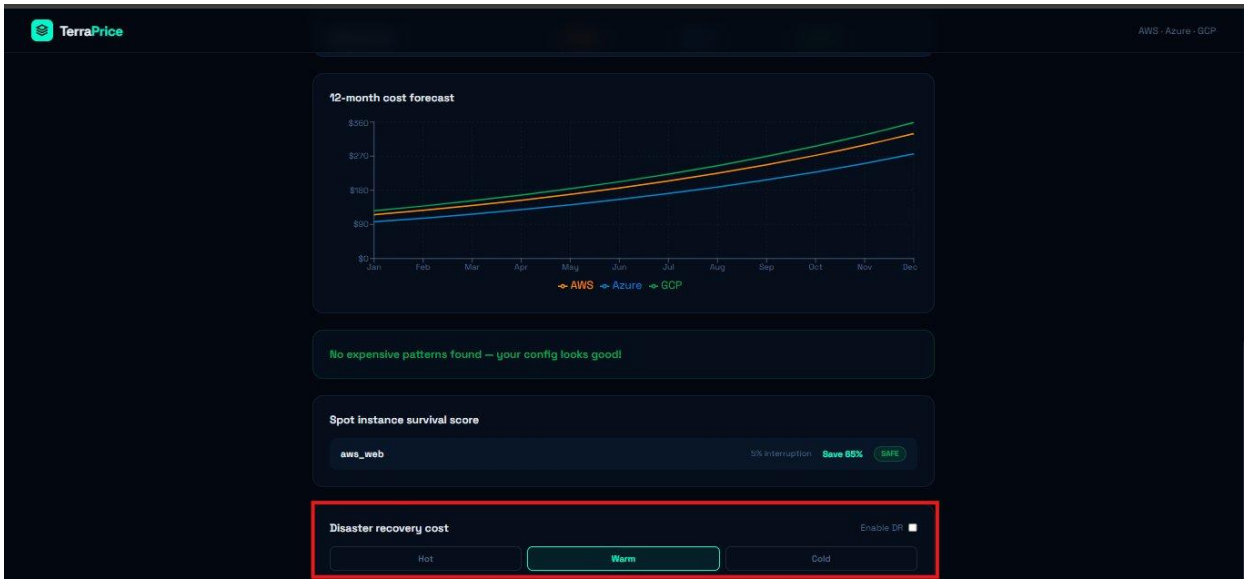

Figure 9: Disaster Recovery cost estimation with Hot, Warm, and Cold DR strategy options

4.7 AI Terraform Generator Output

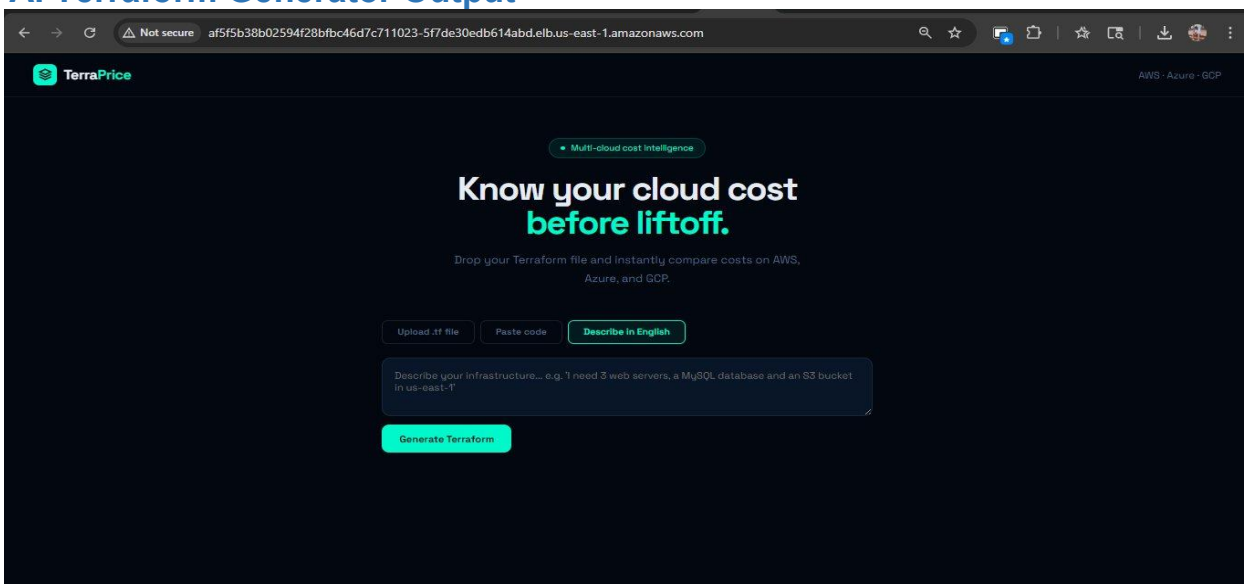

Figure 10: "Describe in English" tab — natural language input for AI Terraform generation

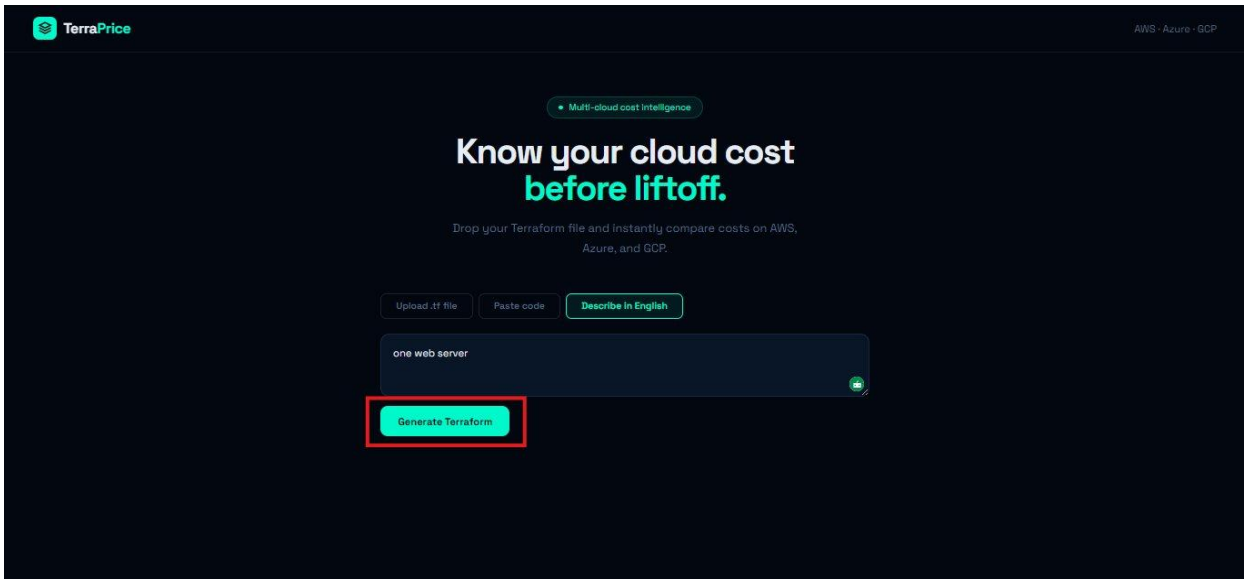

Figure 11: User types "one web server" and clicks Generate Terraform

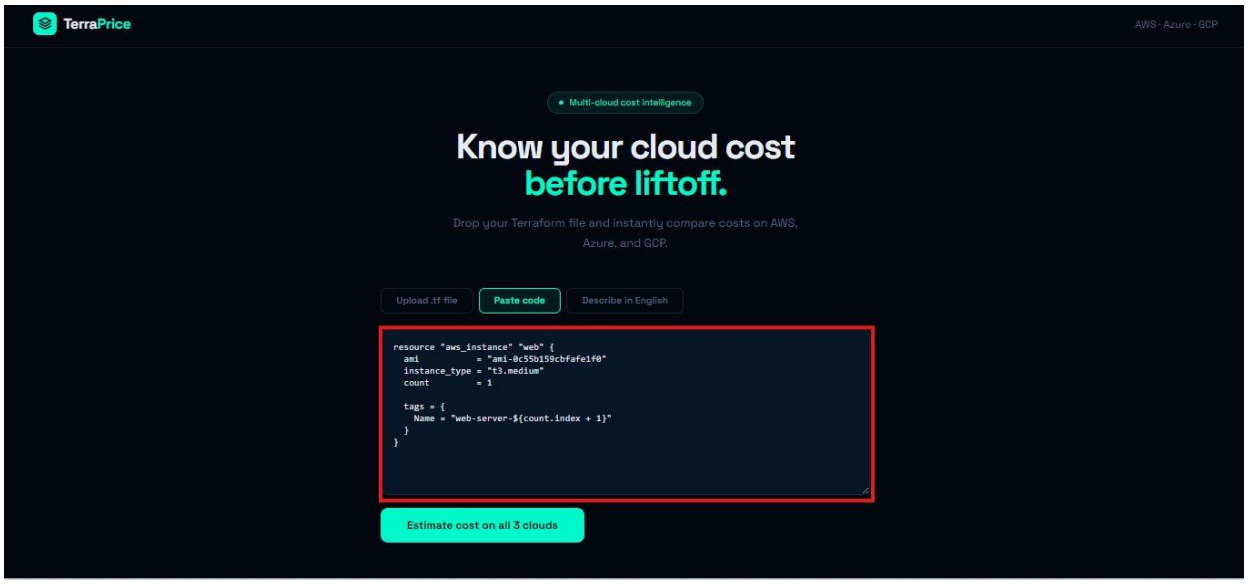

Figure 12: AI-generated Terraform HCL code for an EC2 web server with AMI, instance type, and tags

4.8 Grafana Monitoring Dashboard

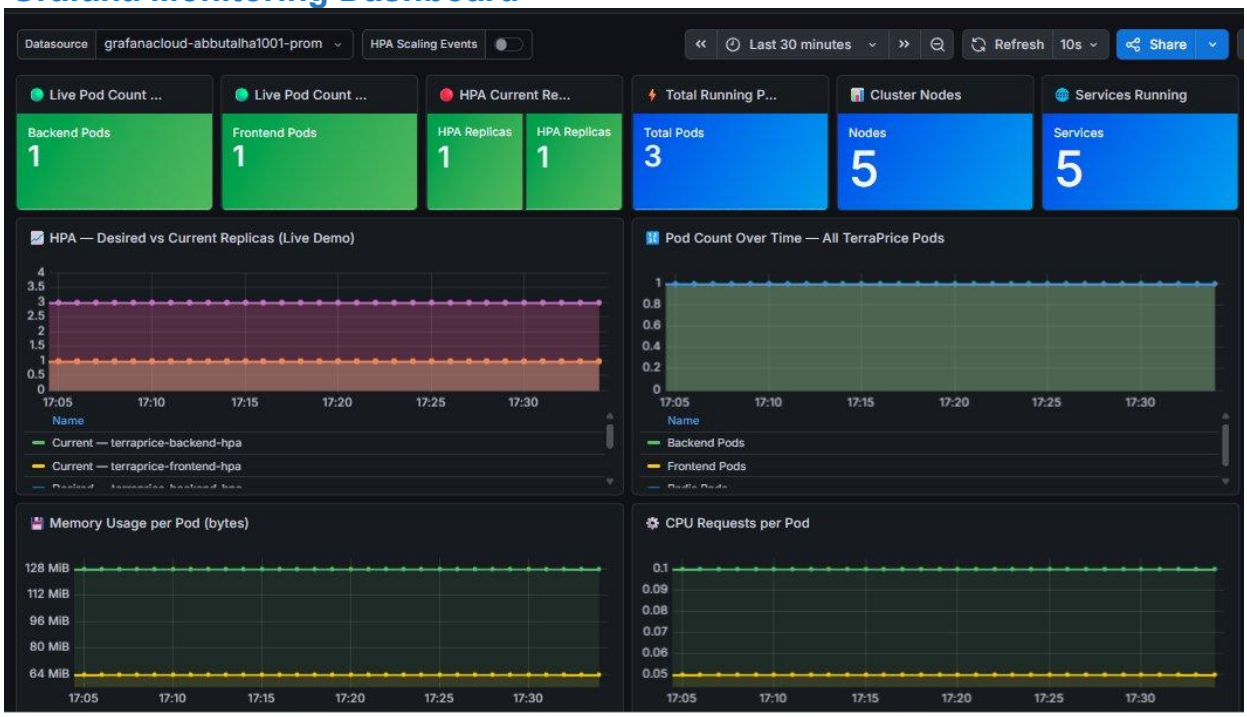

Figure 13: Grafana Cloud live dashboard — Backend Pods: 1, Frontend Pods: 1, Total Pods: 3, Nodes: 5, Services: 5 with HPA and pod count graphs

Figure 14: Grafana — Pod Status (3 Ready), Pod Restarts, and Pod Info Table showing all running pods with node assignments

4.9 CI/CD Pipeline — GitHub Actions Run

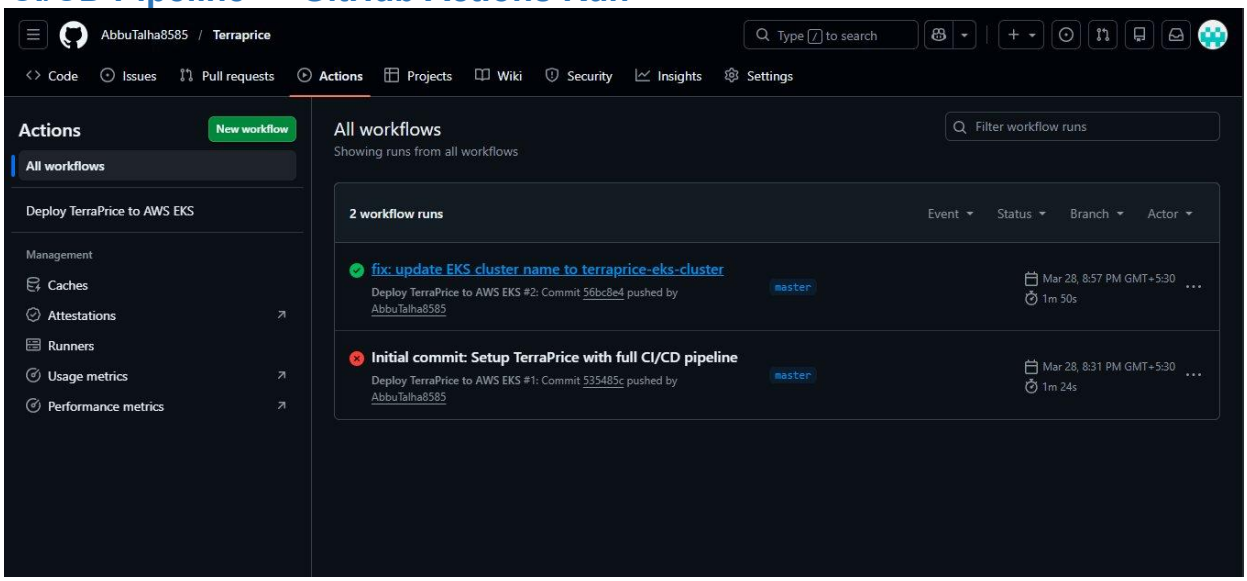

Figure 15: GitHub Actions CI/CD pipeline — "Deploy TerraPrice to AWS EKS" workflow with successful run (green tick) in 1m 50s

5. GitHub Repository & Code

5.1 Repository Details

Repository URL	https://github.com/AbbuTalha8585/Terraprice
Repository Name	Terraprice
Visibility	Public
Author	Abbu Talha
Branch	main

5.2 Repository Structure

Folder / File	Contents
`frontend/`	React + Vite application source code
`backend/`	Python FastAPI application source code
`terraform/`	All Terraform IaC files (.tf) for AWS provisioning
`k8s/`	Kubernetes manifests (Deployments, Services, Ingress, HPA)
`monitoring/`	Grafana and Alloy configuration files
`.github/workflows/`	GitHub Actions CI/CD pipeline YAML files
`docker/`	Dockerfiles for frontend and backend
`docs/`	Architecture diagrams, documentation, and screenshots
`README.md`	Project overview, setup instructions, and deployment guide

5.3 CI/CD Pipeline Steps

1. Checkout code from GitHub repository
2. Run backend unit tests (Python pytest)
3. Build Docker image for backend and push to Amazon ECR
4. Build Docker image for frontend and push to Amazon ECR

5. Configure kubectl to connect to the EKS cluster
6. Apply updated Kubernetes manifests to the EKS cluster
7. Verify deployment rollout status

5.4 Files Included in Repository

- All frontend source code (React components, pages, API hooks)
- All backend source code (FastAPI routes, services, models, parsers)
- Complete Terraform IaC files for all AWS infrastructure
- Kubernetes manifests for all deployments, services, and HPA configs
- Grafana dashboard JSON configuration files
- GitHub Actions workflow YAML files
- Dockerfiles for both frontend and backend
- README.md with full setup, deployment, and usage guide
- Architecture diagram (PNG format)
- This technical documentation (Word format)